Dokumentation - SpectralFreeze

Praxismodul Musikinformatik II

Einleitung **2**
Benutzeroberfläche **2**
Spectral Freeze 2
Spectral Delay 3
Implementierung **4**
Die Klasse FFTProcessor 4
Fazit und Ausblick **8**
References **9**

Einleitung

Im Rahmen dieses Projekts wird ein Audio-Plugin basierend auf dem C++-Framework JUCE konzipiert und entwickelt. Im Kern erfasst das Plugin mittels Fourier-Transformation das aktuelle Spektrum des Eingangssignals und erlaubt es, die spektrale Struktur beizubehalten, zeitlich zu verlängern und es mit dem Ursprungssignal zu vermischen. Das Eingangssignal kann dabei zu selbst gewählten Zeitpunkten eingefroren werden oder automatisch bei einer bestimmten Frequenz. Das eingefrorene Spektrum kann dabei als atmosphärische Klangschicht eingesetzt werden, jedoch kann bei einer hohen Einfrierfrequenz ein glitch-artiger Klangcharakter entstehen.
Des Weiteren gibt es die Möglichkeit, das erfasste Spektrum in Frequenzbänder einzuteilen und diese zeitlich zu versetzen. Dabei stehen klassische Delay-Parameter zur Verfügung, um die spektrale Verzögerung pro Band zu formen. Die einzelnen Bänder müssen dabei aber nicht linear zeitlich versetzt werden, sondern können voneinander unabhängige Delay-Zeit und Feedback-Werte haben. Dadurch kann der glitch-artige Charakter des resultierenden Klangs verstärkt werden. Das gefrorene und das verzögerte Spektrum können in einem selbst gewählten Mischverhältnis miteinander abgestimmt werden.
Um die Auswirkungen des Plugins auf den Klang nachvollziehen zu können, integriert die Benutzeroberfläche ein Spektrogramm. Im folgenden Abschnitt wird vorgestellt, wie die genannten Anforderungen und Funktionalitäten in eine Benutzeroberfläche gegossen werden.

Benutzeroberfläche

Das Plugin lässt sich auf funktionaler Ebene in zwei Bereiche teilen. Die benötigten Parameter und zugehörigen UI-Elemente lassen sich entweder dem *Spectral Freeze* oder dem *Spectral Delay* zuordnen, also entweder dem Beibehalten eines Spektrums oder der zeitliche Verzögerung von Frequenzbändern innerhalb des Spektrums. Zusätzlich gibt es noch den globalen Parameter *dry/wet*, welcher das Mischverhältnis zwischen Original- und verarbeitetem Signal angibt. In den folgenden Teilabschnitten werden entsprechende Funktionsbereiche beschrieben. Das letztendliche GUI-Design ist in Abbildung 1 zu sehen.

Spectral Freeze

Ein zentrales Element auf der Benutzeroberfläche ist der *Freeze*-Button. Dieser Button aktiviert das Einfrieren des Spektrums und bei erneutem Button-Klick ist das Originalsignal wieder hörbar, beziehungsweise nur der *spectral delay*. Im *Retrigger*-Bereich sind Parameter untergebracht, die die automatisierte Auslösung eines erneuten spektralen Einfrierens

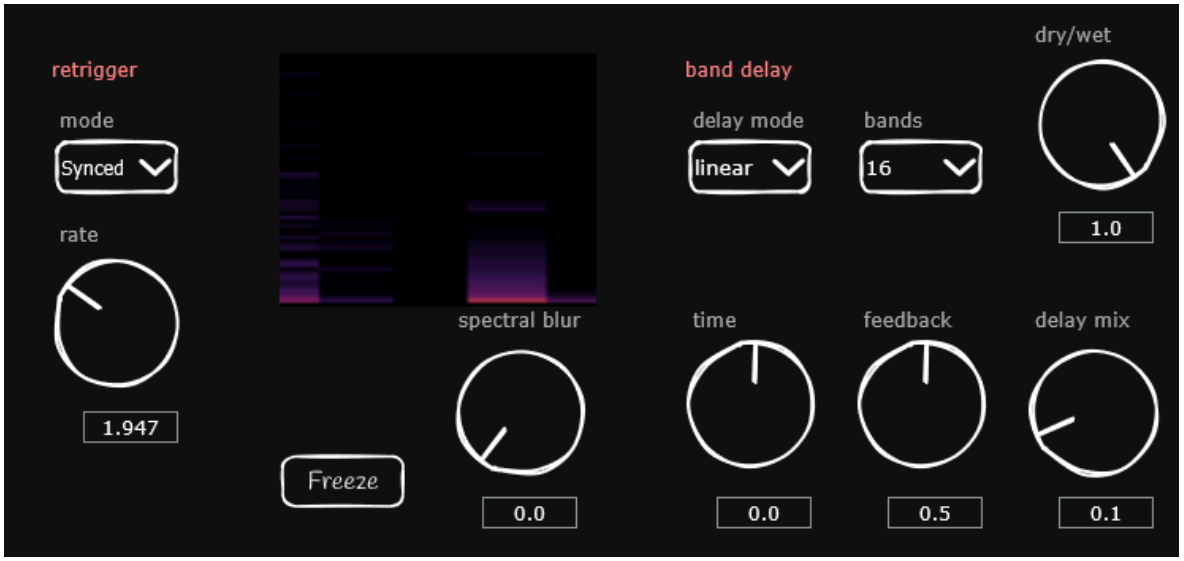

Abbildung 1: Benutzeroberfläche von *SpectralFreeze*

steuern. Ist der Parameter *mode* nicht auf *Live* gestellt, wird in regelmäßigen Abständen in der angegebenen Frequenz ein erneuter *Freeze* ausgelöst. Die Frequenz wird durch den Parameter *rate* angegeben. Dieser bewegt sich im Wertebereich von 0.1Hz bis 20Hz.

Der Parameter *blur* bestimmt zu welchem Anteil die aktuelle Phase mit einem randomisierten Phasenwert interpoliert wird. Der Ansatz die Phase zu randomisieren entspringt dem *"PADSynth" Sound Synthesis Algorithm.* Bei einer zunehmend randomisierten Phase geht die rhythmische Information des Klangs verloren und der Klang erhält einen verwaschenen Charakter. (Nasca, 2011, S.35ff.)

Spectral Delay

Der Bereich des *Spectral Delay* gibt dem Nutzer die Möglichkeit, das Spektrum über den Parameter *bands* in eine selbst gewählte Anzahl an Bändern zu teilen und diese zeitlich zu versetzen.

Mit dem Parameter *delay mode* lässt sich auswählen, auf welche Weise die einzelnen Bänder versetzt werden. Der *delay mode* "linear" bedeutet, dass die Bänder in gleichem Abstand zueinander versetzt werden. Soll das erste Band beispielsweise um zwei Frames versetzt werden, wird das zweite um vier Frames versetzt und alle weiteren Bänder verhalten sich analog dazu und haben jeweils zwei Frames Abstand zueinander. Bei positiven Werten des Parameters *time* werden Bänder mit niedrigeren Frequenzen weniger verzögert als jene mit höheren Frequenzen und bei negativen Werten verhält sich die Delay-Time-Verteilung gespiegelt. Der zweite *delay mode* "random" verteilt die Delay-Zeiten zufällig.

Der Parameter *feedback* steuert die Intensität, mit der das verzögerte Signal wieder in die Delay-Line geschrieben wird.

Delay mix gibt das Mischverhältnis zwischen dem “gefrorenen” Signal und dem verzögerten Signal an.

Implementierung

Die Zustände der auf der Benutzeroberfläche sichtbaren Parameter werden in einem *AudioParameterValueTreeState*-Objekt festgehalten. Sowohl der Audio-Thread, als auch der GUI-Thread haben Zugriff auf dieses Objekt, wodurch die Kommunikation zwischen *PluginEditor* und *PluginProcessor* möglich gemacht wird. (JUCE, 2024)

In der Klasse *SpectralFreezeAudioProcessor* wird die Methode *processBlock* aufgerufen (siehe Abbildung 2). Dort werden die aktuellen Zustände der einzelnen Parameter, in dem die Methode *updateParams* aufgerufen wird.
Die Klasse *FFTProcessor* bildet die Hauptklasse für alle Funktionalitäten, die die spektrale Analyse des aktuellen Signalblocks benötigt. Dazu gehören der *Spectral Freeze* und der *Spectral Delay*.
Jedem Input-Kanal ist ein *FFTProcessor*-Objekt zugeordnet. Die Samples des aktuellen Blocks werden pro Kanal über die Variable *channelData* an die *FFTProcessor*-Objekte weitergegeben. Dort werden weitere Klassen verwendet, die jeweils den *Spectral Freeze* und den *Spectral Delay* umsetzen.

```
updateParams(numSamples, totalNumInputChannels);

for (int channel = 0; channel < totalNumInputChannels; ++channel)
{
    auto* channelData = buffer.getWritePointer (channel);
    fft[channel].processBlock(channelData, numSamples, freezeBypassed, false);
}
```

Abbildung 2: Innerhalb von *processBlock*

Im Folgenden werden die Funktionalitäten der einzelnen Klassen näher beleuchtet.

Die Klasse *FFTProcessor*

Die Klasse *FFTProcessor* sammelt Input-Samples in einem FIFO der Größe 1024 und führt alle 256 Samples eine *Fast Fourier Transform* auf diesen FIFO aus. Die Größe 1024 für den FIFO wurde gewählt, um eine ausreichend hohe Auflösung beizubehalten und die Qualität des Signals bei seiner Rekonstruktion nicht zu verlieren, aber trotzdem performant zu bleiben. Die *hop-size* 256 wurde gewählt, um jeden FFT-Frame zu 75% überlappen zu können. Mit einem Overlap-Add wird sichergestellt, dass keine Diskontinuität entsteht, sobald im FIFO erneut vom Anfang gelesen wird. (Hollemans, 2023)

FFTProccesor enthält einen *input Fifo* und einen *output Fifo.* In den *input Fifo* werden die Samples des eingehenden Signals geschrieben, während die aktuelle Schreibposition mitgeführt wird.
Innerhalb der *processBlock*-Methode, der *FFTProcessor*-Klasse wird die Methode *processSample* aufgerufen, die das Schreiben der Eingangssamples und das Lesen aus dem *output Fifo* umgesetzt (siehe Abblidung 3). Sobald genug Samples gesammelt sind, wird jeweils ein Frame verarbeitet.

```
float FFTProcessor::processSample(float& sample, bool freezeBypassed, bool delayBypassed)
{
    inputFifo[pos] = sample;

    float outputSample = outputFifo[pos];
    outputFifo[pos] = 0.0f;

    pos += 1;
    if (pos == fftSize) {
        pos = 0;
    }

    count += 1;
    if (count == hopSize) {
        count = 0;
        processFrame(freezeBypassed, delayBypassed);
    }

    return outputSample;
```

Abbildung 3: *processSample* in *FFTProcessor*

Dazu werden die Daten aus dem Input-Buffer in einen FFT-Buffer kopiert, sodass darauf die spektrale Analyse erfolgen kann. Der *FFTProcessor* nutzt die FFT-Klasse des Moduls *juce::dsp.* In der Methode *processFrame* wird die Fourier-Transformation mit *performRealOnlyForwardTransform* durchgeführt (siehe Abbildung 4). Dabei werden Audio-Samples gelesen und mit komplexen Zahlen überschrieben, welche die Frequenz-Informationen repräsentieren.
In der darauffolgenden Methode *processSpectrum* kann dann jedes Bin einzeln betrachtet werden. Nachdem das Spektrum verarbeitet und je nach Parameterzustand angepasst wurde, kann das Audiosignal mit *performRealOnlyInverseTransform* wieder rekonstruiert werden.

```
if (!freezeBypassed || !delayBypassed) {
    fft.performRealOnlyForwardTransform(fftPtr, true);
    processSpectrum(fftPtr, numBins, scopedData);
    fft.performRealOnlyInverseTransform(fftPtr);
}
```

Abbildung 4: Ausschnitt aus *processFrame*

In *processSpectrum* werden die komplexen Werte für jedes Bin zunächst in jeweils Magnitude und Phase konvertiert. Die einzelnen Bins werden in Bänder gruppiert, um den *spectral Delay* zu realisieren. Jedem Band wird im Vorfeld eine eigene Delay-Zeit zugewiesen, sodass jedes Band unabhängig von einander zeitlich verschoben werden kann.

Der *FFTProcessor* hält eine Liste von Bändern, beziehungsweise von Instanzen des *structs Band*. Pro Bin wird das entsprechende Band abgerufen, um die verzögerten Samples des aktuellen Bands zu lesen und eingehende Samples in die zugehörigen Delay-Lines zu schreiben.

Das struct *Band* ist in Abbildung 6a zu sehen. Es enthält Informationen darüber, welche Bins es abdeckt, welche Delay-Zeit in Frames, welcher Feedback-Wert und welcher Mix-Wert dem Band zugeordnet ist. Bisher kann ein Band nur eine unabhängige Delay-Zeit haben, während Feedback und Mix für alle Bänder gleich sind. Diese Variablen können jedoch mit der Weiterentwicklung des Plug-ins ebenfalls unabhängig von den anderen Bändern gestaltet werden, um im Endergebnis einen umso komplexeren Klang erhalten zu können.

Die Zuteilung der einzelnen Bins zu den Bändern erfolgt in der Methode *initGrid* (siehe Abbildung 6b). Jedes Band hat repräsentiert eine Gruppe von Bins, wobei die Größe der Bänder logarithmisch verteilt ist, um der unterschiedlich wahrgenommenen Detailtiefe gerecht zu werden und bei einem Delay das Gefühl zu haben, das jeweils gleichgroße Frequenzbänder zeitlich verschoben werden. Bei tiefen Frequenzen sind kleine Frequenzänderungen leicht hörbar, weshalb das repräsentierende Band eher klein sein muss. Bins der höheren Frequenzen können in größeren Bändern zusammengefasst werden.

Zudem hält jedes *Band* ein Objekt des Typs *SpectralDelay*. Dieses Objekt ermöglicht es für jeden Frame, die Frequenzinformationen für jedes Bin im entsprechenden Band zu hinterlegen und stellt analog zu klassischen Delay-Strukturen Schreib- und Lese-Mechanismen zur Verfügung. Das bedeutet, dass jedes Band pro Bin Frequenzinformationen für jedes Frame hinterlegt, das maximal eine Sekunde in der Vergangenheit liegt.
In der Methode *processSpectrum* kann dann pro Bin auf das entsprechende *Band* zugegriffen werden, um den *Spectral Delay* zu realisieren.

Um den *Spectral Freeze* zu realisieren, werden die Magnituden pro Bin in dem Array *frozenMagnitudes* festgehalten. Bei Anforderung des *Spectral Freeze* werden die nächsten eingehenden Magnituden mit den zu dem Zeitpunkt aktuellen Magnituden so lange überschrieben, bis der Freeze entweder durch manuellen Klick des Buttons oder durch den Retrigger-Mechanismus beendet wird.

Zum Schluss wird die ausgehende Magnitude mit dem Ergebnis der Verarbeitung in gegebenem Mischverhältnis addiert. Die ausgehende Phase ergibt sich aus der Interpolation von eingehender Phase und zufällig generierter Phase.

Für das Spektrogramm werden die Magnitudenwerte in Dezibel umgerechnet, um in jedem Frame die Energie der einzelnen Bins farblich darstellen zu können.

```cpp
243 void FFTProcessor::processSpectrum(float* data, int numBins, std::vector<float>& scopedData)
244 {
245     auto* cdata = reinterpret_cast<std::complex<float>*>(data);
246     Band* currentBand = getBandIndex(1);
247
248     for (int i = 0; i < numBins; ++i) {
249         float magnitude = std::abs(cdata[i]);
250         float phase = std::arg(cdata[i]);
251
252         Band* newBand = getBandIndex(i + 1);
253
254         if (currentBand != newBand) {
255             currentBand->delay.advance();
256             if (currentBand->delay.delayTimeChanging) currentBand->delay.fadeCounter--;
257             currentBand = newBand;
258         }
259
260         float delayed = 0.0f;
261         if (currentBand) {
262
263             if (freezeEnabled) {
264                 magnitude = frozenMagnitudes[i];
265             }
266             if (delayEnabled) {
267                 if (currentBand->delay.delayTimeChanging) {
268                     delayed = currentBand->delay.crossfadeDelayFrames(i, currentBand->binStart);
269                 }
270                 else {
271                     delayed = currentBand->delay.read(i, currentBand->binStart, currentBand->delayFrames);
272                 }
273             }
274
275             float fb = delayed * currentBand->feedback;
276             float wet = magnitude + fb;
277
278             if (triggerFreeze ) {
279                 frozenMagnitudes[i] = magnitude;
280             }
281
282             if (delayEnabled) {
283                 currentBand->delay.write(i, currentBand->binStart, wet);
284             }
285
286             float outMag = ((1.f - currentBand->mix) * magnitude) + (currentBand->mix * fb);
287             float randPhase = r.nextFloat() * 2.0f * float_Pi;
288             float outPhase = blurPhase(randPhase, phase, phaseBlur);
289
290             cdata[i] = std::polar(outMag, outPhase);
291         }
292         //for spectrogram
293         scopedData[i] = juce::Decibels::gainToDecibels(magnitude) - juce::Decibels::gainToDecibels(numBins);
294     }
295     if (triggerFreeze) {
296         freezeEnabled = true;
```

Abbildung 5: Ausschnitt aus *processSpectrum*

```
struct Band {
    //fft bin range
    int binStart = 0;
    int binEnd = 0;

    //delay
    int delayFrames = 0;
    float feedback = 0.0f;
    float mix = 0.5f;

    SpectralDelay delay;
};
```

Abbildung 6a: struct *Band*

```
void FFTProcessor::initGrid(int bandCountIndex, float delayTime,
{
    gridEnabled = true;
    fMax = sr / 2.;
    numBands = bandCountValues[bandCountIndex];
    float ratio = pow(fMax / fMin, 1.0f / numBands);
    float delayTimePerBand = std::abs(delayTime) / numBands;

    bands.clear();

    for (int i = 0; i < numBands; ++i) {
        Band b;
        float f0 = fMin * pow(ratio, i);
        float f1 = fMin * pow(ratio, i + 1);

        b.binStart = ceil(f0 * fftSize / sr);
        b.binEnd = floor(f1 * fftSize / sr);

        int maxDelayFrames = sr / hopSize;
        int bandSize = b.binEnd - b.binStart + 1;

        b.delay.prepare(maxDelayFrames, bandSize);
        if (b.binStart <= b.binEnd) bands.push_back(b);

    }
    setFeedback(feedback);
    setDelayMix(delayMix);
    setDelayTime(delayTime, sr, delayMode);
}
```

Abbildung 6b: Methode *initGrid*

Fazit und Ausblick

Das entwickelte Plugin setzt Funktionalitäten um, die sich in zwei Effektgruppen aufteilen lassen. Die erste Funktionalitätsgruppe realisiert den *Spectral Freeze,* also das “Einfrieren” des aktuellen Spektrums. Die zweite Gruppe umfasst den *Spectral Delay*. Dabei können einzelne Frequenzbänder auf verschiedene Weise unabhängig voneinander zeitlich versetzt werden. Durch die Verwendung beider Effekte in einem Kontext kann ausgehend vom Eingangssignal ein zugleich atmosphärischer und glitch-artiger Klang entstehen, dessen Charakter sich durch den Nutzer formen lässt.
Bestimmte Aspekte lassen sich in weiteren Iterationen in der Entwicklung noch weiter ausarbeiten. Zum Beispiel lässt sich die Größe des FFT-Buffers nicht ändern und hat eine festgelegte Größe, die sich aus einem Kompromiss zwischen Performance und Auflösung ergibt. Diese Konstante zieht sich durch nahezu alle Funktionalitäten des Plugins, da sie sich alle auf das Ergebnis der spektralen Analyse beziehen. Als Nutzer könnte die Möglichkeit diese Größe zu steuern jedoch von Mehrwert sein, falls dieser beispielsweise Wert auf eine hohe Klangqualität legt oder die Wahl einer niedrigen Auflösung aus ästhetischen Beweggründen zeugt.
Des Weiteren wurde die Phase eines Bins in der spektralen Verarbeitung nur wenig berücksichtigt und nicht zusammen mit der zugehörigen Magnitude zusammen betrachtet. Dies stammt aus der Annahme, dass die Phase eines Bins in der menschlichen Wahrnehmung weniger Bedeutung für den Klang hat als die Magnitude. Die Möglichkeit, die Phase eines Bins im *Spectral Freeze* und *Delay* mitführen zu können, würde eine Veränderung im Timbre und Rhythmus des verarbeiteten Ergebnis bedeuten. In der Implementierung muss dabei jedoch auf die dadurch entstehenden Diskontinuitäten im Signal Rücksicht genommen werden.

References

Hollemans, M. (2023, September 5). *FFT Processing in JUCE*. audiodev.blog. Retrieved März 12, 2026, from https://audiodev.blog/fft-processing/

JUCE. (2024). *Tutorial: Adding plug-in parameters*. JUCE. Retrieved March 11, 2026, from https://juce.com/tutorials/tutorial_audio_parameter/

Nasca, P. (2011). “PADSYNTH” SOUND SYNTHESIS ALGORITHM. *STUDIA UNIV. BABES¸–BOLYAI, INFORMATICA*, *LVI*(1), 35-43. https://www.cs.ubbcluj.ro/~studia-i/contents/2011-1/04-Nasca.pdf